🗂️ Section 7 — Volumes and the Filesystem Model (Concise)

This section explains how container filesystems work, why container data is normally not persistent, and how Docker volumes provide safe, durable storage.
It also clarifies how data can live on the host or on remote systems.

7.1 Container Filesystems: Ephemeral by Design

A container's root filesystem:

- Comes from immutable **image layers**
- Adds a thin, writable **container layer** on top
- The writable layer is **temporary**
    - Deleted when the container is removed (unless stored elsewhere)

Implication:

> Anything written inside the container's default filesystem disappears when the container is deleted.

This behavior is intentional—containers are designed to be disposable.

7.2 Why Data Must Be Stored Outside the Container

To persist data across container runs, data must live:

1. **On the host filesystem**, or
2. **On a remote storage system**, such as:
    - NFS shares
    - Cloud storage (EBS, Ceph, S3 gateways)
    - SAN/NAS devices

Docker solves this with **volumes** and **bind mounts**.

7.3 Two Ways to Persist Data

1. Docker Volumes (Preferred)

- Managed by Docker
- Stored under `/var/lib/docker/volumes`
- Decoupled from the host directory structure
- Good for databases and long-lived data

```
docker volume create mydata
docker run -v mydata:/data myimage
```

Inside the container, `/data` persists across runs and redeployments.

2. Bind Mounts (Direct Host Path)

- Directly maps a host directory into the container
- Useful for development, config files, or explicit data paths

```
docker run -v /host/path:/container/path myimage
```

Anything written inside `/container/path` appears on the host at `/host/path`.

OS concept:

> The mount namespace isolates filesystem views, but bind mounts selectively reintroduce host paths into the container's namespace.

7.4 Remote Persistence (Short Overview)

Persistence **does not require networking**, but **remote persistence** may.

Three common models:

A. Local Host Storage (No Networking)

- Most basic and common
- Docker volumes or bind mounts
- Data stays on that machine

B. Network Filesystems (Requires Networking)

Examples:

- NFS
- SMB

7.5 Example: Persistent Database with a Volume

```
docker volume create dbdata

docker run \
  -d \
  --name mydb \
  -v dbdata:/var/lib/postgresql/data \
  postgres:16
```

Data persists even if:

- the container is stopped
- removed
- replaced with a new version

OS perspective:

> Database writes bypass the ephemeral container layer and go directly to stable

7.6 Summary of Commands for Section 7

```
# Create a named volume
docker volume create mydata

# Use a volume inside a container
docker run -v mydata:/data myimage

# Bind mount a host directory
docker run -v /host/path:/container/path myimage

# List volumes
docker volume ls

# Inspect a specific volume
docker volume inspect mydata
```

7.7 Key Takeaways

- Container root filesystems are **immutable + temporary**.
- Persistence requires data to live **outside** the container layer.
- Docker provides:
  - **Volumes** (managed, safer)
  - **Bind mounts** (direct host access)
- Remote storage is handled by the *host*, then mounted into containers.
- This integrates cleanly with OS-level mount namespaces and real-world networking/storage setups.